simpler equations directly relating the pole-zero positions to the recursion coefficients. For example, a system containing two poles and two zeros, called as **biquad**, has the following relations:

EQUATION 33-5
Biquad design equations. These equations give the recursion coefficients, a_0, a_1, a_2, b_1, b_2, from the position of the poles: r_p & ω_p, and the zeros: r_0 & ω_0.

$$\begin{aligned} a_0 &= 1 \\ a_1 &= -2r_0\cos(\omega_0) \\ a_2 &= r_0^2 \\ b_1 &= 2r_p\cos(\omega_p) \\ b_2 &= -r_p^2 \end{aligned}$$

After the transfer function has been specified, how do we find the frequency response? There are three methods: one is mathematical and two are computational (programming). The mathematical method is based on finding the values in the z-plane that lie on the unit circle. This is done by evaluating the transfer function, $H(z)$, at $r = 1$. Specifically, we start by writing down the transfer function in the form of either Eq. 33-3 or 33-4. We then replace each z with $e^{-j\omega}$ (that is, $re^{-j\omega}$ with $r = 1$). This provides a mathematical equation of the frequency response, $H(\omega)$. The problem is, the resulting expression is in a very inconvenient form. A significant amount of algebra is usually required to obtain something recognizable, such as the magnitude and phase. While this method provides an exact equation for the frequency response, it is difficult to automate in computer programs, such as needed in filter design packages.

The second method for finding the frequency response also uses the approach of evaluating the z-plane on the unit circle. The difference is that we only calculate *samples* of the frequency response, not a mathematical solution for the entire curve. A computer program loops through, perhaps, 1000 equally spaced frequencies between $\omega = 0$ and $\omega = \pi$. Think of an ant moving between 1000 discrete points on the upper half of the z-plane's unit circle. The magnitude and phase of the frequency response are found at each of these location by evaluating the transfer function.

This method works well and is often used in filter design packages. Its major limitation is that it does not account for *round-off noise* affecting the system's characteristics. Even if the frequency response found by this method looks perfect, the implemented system can be completely unstable!

This brings up the third method: find the frequency response from the recursion coefficients that are actually used to implement the filter. To start, we find the impulse response of the filter by passing an impulse through the system. In the second step, we take the DFT of the impulse response (using the FFT, of course) to find the system's frequency response. The only critical item to remember with this procedure is that enough samples must be taken of the impulse response so that the discarded samples are *insignificant*. While books